

SmartSenseCNC — Final Execution Plan

Real-Time CNC Downtime Detection using ESP32 + MPU6050 + SCT-013 + Machine Learning

Team: Tech Titans

Duration: 6 Days

Project Goal: Build a low-cost Industrial IoT system capable of detecting CNC machine states (OFF, IDLE, ACTIVE) using vibration and current sensing with Machine Learning-based classification and downtime alerting.

1. Final Project Objective

The project focuses on developing a real-time CNC downtime monitoring system using: ESP32 (MYOSA Board) MPU6050 Vibration Sensor SCT-013 Current Sensor Buzzer for downtime alerts The system will: Collect sensor readings Compute RMS values Train a Random Forest ML model Predict machine states in real time Trigger alerts during unnecessary idle time Display analytics on a live dashboard

2. Final Sensor Selection

ESP32: Main controller, WiFi communication, and data processing.

MPU6050: Detects machine vibration and differentiates IDLE vs ACTIVE states.

SCT-013: Measures machine current consumption and differentiates OFF vs ON states.

Buzzer: Activates during extended machine downtime to indicate electricity waste.

3. Final System Architecture

MPU6050 + SCT-013 → ESP32 → RMS Feature Extraction → WiFi Transmission → Flask Backend → Random Forest ML Model → Dashboard + Buzzer Alert

4. Machine State Definitions

OFF: No current and no vibration.

IDLE: Machine powered ON but no productive operation. Current present, vibration low.

ACTIVE: Machine operational with current and strong vibration indicating machining activity.

5. Downtime Alert Logic

If the system predicts the machine state as IDLE for more than the defined threshold duration (recommended: 30–60 seconds for demo), the buzzer activates and the dashboard displays a downtime alert.

6. Demo Simulation Strategy

Since continuous access to a CNC machine is limited, a simulated rig will be used: **OFF:** Washing machine OFF + vibration motor OFF **IDLE:** Washing machine ON + vibration motor OFF **ACTIVE:** Washing machine ON + vibration motor ON

7. Feature Engineering

The ML model will use processed RMS values instead of raw sensor values. Current RMS Vibration RMS features provide: Better stability Noise reduction Improved ML classification accuracy

8. Final Dataset Structure

The dataset will contain: current_rms vibration_rms state

9. Machine Learning Model

Algorithm: **Random Forest Classifier**

Reasons for selection: Works effectively on small datasets Handles noisy sensor data well Fast training and prediction Reliable for structured sensor datasets

10. Live System Workflow

ESP32 Tasks: Read sensors Compute RMS values Send processed data to backend **Backend Tasks:** Receive sensor data Run ML prediction Update dashboard Track downtime duration Trigger buzzer alerts

11. Dashboard Features

Live Machine State Current RMS Graph Vibration RMS Graph Downtime Timer Idle Alerts Historical Logs CSV Export

12. Team Responsibilities

Member 1 — Embedded & Sensors ESP32 programming Sensor integration RMS calculations Buzzer control
Member 2 — Backend & Dashboard Flask backend Database integration Dashboard development
Member 3 — Machine Learning Dataset preparation Random Forest training Prediction pipeline integration

13. 6-Day Execution Timeline

Day 1: Hardware setup and sensor verification

Day 2: RMS processing implementation

Day 3: Dataset collection and labeling

Day 4: ML model training and evaluation

Day 5: Backend integration and live prediction testing

Day 6: Final testing, debugging, dashboard polishing, and demo recording

14. Why Machine Learning?

Traditional rule-based systems require manual threshold tuning for every machine. Machine Learning allows the system to learn operational behavior directly from sensor data and adapt more effectively to different machine conditions.

15. Final Expected Outcome

The final system will: Detect OFF / IDLE / ACTIVE machine states Identify downtime automatically Trigger downtime alerts using buzzer notifications Provide live ML-based predictions Display real-time analytics and logs Demonstrate Industry 4.0 smart monitoring concepts